

SQL SUBQUERIES

MASTERY GUIDE

Scalar · Correlated · Inline Views · CTEs · EXISTS · Lateral

From Your Senior Developer — Master Every Subquery Pattern

SUB

20 Patterns · 10 Pitfalls · Performance Tactics · CTE Mastery

Complete with SQL Examples, Anti-patterns & Query Optimisation

TABLE OF CONTENTS

■ 1. Foundations of Subqueries

■ What is a Subquery · Execution Model · Placement Rules

■ 2. Types of Subqueries

■ Scalar · Row · Table · Correlated · Non-Correlated

■ 3. WHERE Clause Subqueries

■ IN · NOT IN · EXISTS · NOT EXISTS · Comparison Operators

■ 4. FROM Clause — Inline Views & Derived Tables

■ Derived Tables · Aggregation Layers · Multi-level Nesting

■ 5. SELECT Clause — Scalar Subqueries

■ Computed Columns · Conditional Scalars · Correlated Scalars

■ 6. CTEs — Common Table Expressions

■ Simple CTEs · Recursive CTEs · Chained CTEs · Materialisation

■ 7. Correlated Subqueries — Deep Dive

■ Row-by-Row Logic · EXISTS vs JOIN · Performance

■ 8. 20 Tactical Patterns

■ Every Real-World Pattern Solved with Subqueries

■ 9. Classic Pitfalls & Fixes

■ 10 Deadly Mistakes and How to Avoid Them

■ 10. Performance Tactics & Cheat Sheet

■ Optimisation · Decision Guide · Quick Reference

Foundations of Subqueries

What Is a Subquery?

A subquery (also called an inner query or nested query) is a `SELECT` statement embedded inside another SQL statement. The outer query uses the result of the inner query as a value, a set of values, or a virtual table. Subqueries are the foundation of expressive, composable SQL — they let you break a hard problem into smaller, independently understandable pieces.

Anatomy of a Subquery

```
SELECT  outer_column
FROM    outer_table
WHERE   outer_column OPERATOR (
        SELECT  inner_column      -- the subquery
        FROM    inner_table
        WHERE   inner_condition
      );
-- The subquery runs first (usually), producing a result.
-- The outer query then uses that result.
```

Where Can Subqueries Live?

Clause	Subquery Returns	Example Use-Case
SELECT	Single scalar value	Compute a KPI column per row
FROM	Virtual table (inline view)	Pre-aggregate before joining
WHERE	Scalar, list, or boolean	Filter by dynamic criteria
HAVING	Scalar value	Filter groups by aggregate comparison
INSERT INTO ... SELECT	Table of rows	Copy filtered data
UPDATE ... SET	Scalar value	Set a value from another table
JOIN ... ON (subquery)	Table (derived)	Complex join conditions

★ Senior Dev Principle #1

Subqueries and JOINS are often interchangeable — but not always equal in performance. Write subqueries for CLARITY first. Refactor to JOINS or CTEs when the query plan shows a bottleneck. Readable code saves debugging hours at 2am.

Types of Subqueries

Type	Returns	Correlated?	Used In	Example Scenario
Scalar	Exactly 1 row, 1 col	Either	SELECT, WHERE, HAVING	Get max salary in dept
Row	Exactly 1 row, N cols	Either	WHERE (a,b) = (sq)	Match composite key
Table (Inline View)	Multiple rows & cols	No	FROM clause	Pre-aggregate orders
Column List	Multiple rows, 1 col	Either	IN / NOT IN / ANY / ALL	Get list of active IDs
Correlated	Depends on outer row	Yes	WHERE, SELECT, HAVING	Salary vs dept avg
Non-Correlated	Fixed result set	No	Anywhere	Static lookup values

Correlated vs Non-Correlated

```
-- NON-CORRELATED: runs ONCE, result reused for all outer rows
SELECT name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
-- The inner SELECT AVG(...) executes a single time.

-- CORRELATED: re-executes for EACH outer row (references outer alias)
SELECT e.name, e.salary, e.dept_id
FROM employees e
WHERE e.salary > (
    SELECT AVG(salary)
    FROM employees
    WHERE dept_id = e.dept_id -- <-- references outer table alias
);
-- Runs once per employee row. Can be slow on large tables.
```

! Correlated Subquery Performance

A correlated subquery in WHERE executes N times (once per outer row). For 1 million rows that is 1 million extra queries. Always check if a window function or JOIN can replace it.

WHERE Clause Subqueries

Comparison Operators with Scalar Subquery

```
-- Employees earning more than the company average
SELECT name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);

-- Most recently hired employee
SELECT * FROM employees
WHERE hire_date = (SELECT MAX(hire_date) FROM employees);

-- IMPORTANT: if subquery returns > 1 row, this crashes!
-- Use MAX/MIN/TOP 1 to guarantee a scalar result.
```

IN / NOT IN with Column Subquery

```
-- Customers who placed an order in the last 30 days
SELECT *
FROM customers
WHERE id IN (
    SELECT DISTINCT customer_id
    FROM orders
    WHERE created_at >= CURRENT_DATE - INTERVAL '30 days'
);

-- Products never ordered (NOT IN – safe version with NULL guard)
SELECT * FROM products
WHERE id NOT IN (
    SELECT product_id FROM order_items WHERE product_id IS NOT NULL
);
```

! NOT IN + NULL = Zero Rows

If the subquery returns ANY NULL, NOT IN returns zero rows silently. Always add WHERE col IS NOT NULL inside the subquery, or use NOT EXISTS instead (NULL-safe by design).

EXISTS / NOT EXISTS

```
-- EXISTS: true if subquery returns at least one row
SELECT c.*
FROM   customers c
WHERE  EXISTS (
    SELECT 1          -- SELECT 1, SELECT *, column – all equivalent
    FROM   orders o
    WHERE  o.customer_id = c.id
    AND    o.status = 'completed'
);

-- NOT EXISTS: true if subquery returns ZERO rows (NULL-safe!)
SELECT p.*
FROM   products p
WHERE  NOT EXISTS (
    SELECT 1 FROM order_items oi WHERE oi.product_id = p.id
);

-- EXISTS short-circuits: stops scanning at first match. Very fast.
```

ANY / ALL / SOME Operators

```
-- ANY: true if condition holds for at least one value in the list
SELECT * FROM products
WHERE price > ANY (SELECT price FROM products WHERE category='Budget');
-- Means: price higher than at least one budget product

-- ALL: true if condition holds for every value in the list
SELECT * FROM products
WHERE price > ALL (SELECT price FROM products WHERE category='Budget');
-- Means: price higher than EVERY budget product (i.e., the maximum)
-- SOME is a synonym for ANY in standard SQL
```

Row Subquery (Multi-Column Match)

```
-- Match on composite key in one clean expression
SELECT *
FROM   inventory
WHERE  (warehouse_id, product_id) IN (
    SELECT warehouse_id, product_id
    FROM   reorder_list
    WHERE  priority = 'HIGH'
);

-- Supported in: PostgreSQL, MySQL 5.7+, SQLite 3.15+
-- SQL Server: use EXISTS with AND instead
```

Derived Tables (Inline Views)

A subquery in the FROM clause is called a derived table or inline view. It behaves exactly like a real table — you can JOIN it, filter it, aggregate over it. This is the primary tool for **breaking complex aggregations into layers**.

```
-- Layer 1: aggregate orders per customer
-- Layer 2: find customers above average order value
SELECT  outer_q.customer_id, outer_q.avg_order
FROM (
    SELECT  customer_id,
            COUNT(*)      AS order_count,
            AVG(total)     AS avg_order
    FROM    orders
    GROUP BY customer_id
) AS outer_q
WHERE  outer_q.avg_order > 500
ORDER BY outer_q.avg_order DESC;
```

Multi-Level Nesting

```
-- Find departments where the top earner is above 100k
SELECT  dept_name
FROM (
    SELECT  d.dept_name,
            MAX(e.salary) AS max_salary
    FROM    employees e
    JOIN    departments d ON e.dept_id = d.id
    GROUP BY d.dept_name
) AS dept_summary
WHERE  max_salary > 100000;
-- Senior tip: flatten this with a CTE for readability (Chapter 6)
```

Joining Two Derived Tables

```
-- Compare this year vs last year revenue by region
SELECT  cy.region,
        cy.revenue    AS current_year,
        py.revenue    AS prior_year,
        cy.revenue - py.revenue AS growth
FROM (
    SELECT region, SUM(amount) AS revenue
    FROM   sales WHERE YEAR(sale_date) = 2024 GROUP BY region
) AS cy
LEFT JOIN (
    SELECT region, SUM(amount) AS revenue
    FROM   sales WHERE YEAR(sale_date) = 2023 GROUP BY region
) AS py ON cy.region = py.region
ORDER BY growth DESC;
```

✓ Pro Tip — Derived Tables vs CTEs

Derived tables live inside the query and cannot be reused. CTEs (WITH clause) are named and can be referenced multiple times in the same query. Prefer CTEs for readability whenever a derived table is used more than once.

SELECT Clause — Scalar Subqueries

Computed Columns via Scalar Subquery

A scalar subquery in the SELECT clause adds a computed column derived from another table. It MUST return exactly one row and one column — or it crashes. NULL is returned if the subquery finds no rows.

```
-- Show each employee with their department's average salary
SELECT  e.name,
        e.salary,
        (
            SELECT AVG(salary)
            FROM    employees e2
            WHERE   e2.dept_id = e.dept_id    -- correlated!
        ) AS dept_avg,
        e.salary - (
            SELECT AVG(salary) FROM employees e2 WHERE e2.dept_id = e.dept_id
        ) AS vs_avg
FROM    employees e;

-- Tip: this runs the subquery TWICE per row. Use window function instead:
-- AVG(salary) OVER (PARTITION BY dept_id) AS dept_avg
```

Conditional Scalar Subquery

```
-- Add latest order date and total order count per customer
SELECT  c.name,
        c.email,
        (SELECT MAX(created_at) FROM orders WHERE customer_id = c.id)
          AS last_order_date,
        (SELECT COUNT(*) FROM orders WHERE customer_id = c.id)
          AS total_orders
FROM    customers c
WHERE   c.active = 1;

-- Performance note: two correlated subqueries per row.
-- Refactor with a single LEFT JOIN + GROUP BY for large tables.
```

Scalar Subquery in CASE Expression

```
-- Classify each product's stock vs average
SELECT  p.name,
        p.stock,
        CASE
            WHEN p.stock > (SELECT AVG(stock) FROM products) THEN 'Above Average'
            WHEN p.stock = (SELECT AVG(stock) FROM products) THEN 'At Average'
            ELSE 'Below Average'
        END AS stock_status
FROM    products  p;

-- Better: compute AVG once in a CTE, reference it three times cleanly.
```

What Is a CTE?

A CTE (WITH clause) is a named, temporary result set that exists only for the duration of one query. Think of it as giving a subquery a name. CTEs make complex queries **readable, testable, and maintainable**. They are the senior developer's preferred tool over deeply nested subqueries.

```
-- Basic CTE syntax
WITH cte_name AS (
    SELECT col1, col2
    FROM   some_table
    WHERE  condition
)
SELECT *
FROM     cte_name
WHERE    col1 > 100;

-- Multiple CTEs (chained with comma)
WITH
    cte_a AS (SELECT ...),
    cte_b AS (SELECT ... FROM cte_a ...),
    cte_c AS (SELECT ... FROM cte_b ...)
SELECT * FROM cte_c;
```

CTE vs Subquery — The Real Difference

Feature	Subquery	CTE
Reusability in same query	No — must repeat	Yes — reference by name
Readability	Decreases with nesting	Linear, named steps
Debuggability	Hard to isolate	Run CTE block alone
Recursion support	No	Yes (RECURSIVE)
Performance	Same or better*	Same or better*
ORDER in result	Only outer query	Only outer query

Recursive CTEs — Walking Hierarchies

```
-- Walk an employee org chart to any depth
WITH RECURSIVE org_chart AS (
  -- Anchor: root nodes (no manager)
  SELECT id, name, manager_id, 0 AS depth,
         CAST(name AS VARCHAR(500)) AS path
  FROM   employees
  WHERE  manager_id IS NULL
  UNION ALL
  -- Recursive: join each child to its parent
  SELECT e.id, e.name, e.manager_id,
         oc.depth + 1,
         oc.path || ' > ' || e.name
  FROM   employees e
  INNER JOIN org_chart oc ON e.manager_id = oc.id
)
SELECT  depth, path, name
FROM    org_chart
ORDER BY path;
```

Chained CTEs for Step-by-Step Analysis

```
WITH
-- Step 1: aggregate raw orders
order_summary AS (
    SELECT  customer_id,
            COUNT(*)   AS order_count,
            SUM(total) AS lifetime_value
    FROM    orders
    WHERE   status = 'completed'
    GROUP BY customer_id
),
-- Step 2: classify customers
customer_segments AS (
    SELECT  customer_id, order_count, lifetime_value,
            CASE
                WHEN lifetime_value >= 10000 THEN 'Platinum'
                WHEN lifetime_value >= 5000  THEN 'Gold'
                WHEN lifetime_value >= 1000  THEN 'Silver'
                ELSE 'Bronze'
            END AS segment
    FROM    order_summary
),
-- Step 3: count per segment
segment_counts AS (
    SELECT  segment, COUNT(*) AS customers
    FROM    customer_segments
    GROUP BY segment
)
SELECT * FROM segment_counts ORDER BY customers DESC;
```

Correlated Subqueries — Deep Dive

How Correlated Subqueries Execute

For every row the outer query considers, the database engine re-evaluates the inner query using that row's column values. This is powerful — but expensive. Understanding the execution model is the difference between a 50ms query and a 50-second query.

```
-- Find employees earning above their own department's average
SELECT  e1.name, e1.salary, e1.dept_id
FROM    employees e1
WHERE   e1.salary > (
    SELECT  AVG(e2.salary)
    FROM    employees e2
    WHERE   e2.dept_id = e1.dept_id  -- outer reference
);
-- Execution: for employee #1 (dept=10) -> AVG of dept 10
--             for employee #2 (dept=20) -> AVG of dept 20
--             ... N total executions for N employees
-- BETTER: window function – computes once, applied per row
SELECT  name, salary, dept_id
FROM (
    SELECT  name, salary, dept_id,
            AVG(salary) OVER (PARTITION BY dept_id) AS dept_avg
    FROM    employees
) sub
WHERE salary > dept_avg;
```

Correlated UPDATE and DELETE

```
-- UPDATE: set each product's category_avg_price field
UPDATE products p
SET    p.category_avg = (
    SELECT AVG(price) FROM products p2 WHERE p2.category = p.category
)
WHERE  p.active = 1;

-- DELETE: remove orders older than the customer's average order gap
DELETE FROM orders o
WHERE  o.created_at < (
    SELECT MIN(created_at) FROM orders o2 WHERE o2.customer_id = o.customer_id
) -- removes any order before the customer's first (none – example pattern)

-- Real use: delete duplicate rows keeping latest
DELETE FROM logs
WHERE  id NOT IN (
    SELECT MAX(id) FROM logs GROUP BY session_id
);
```

20 Tactical Patterns — Crack Every Problem

Pattern 1 — Scalar Benchmark Comparison

Problem: Compare each row against a single global metric (avg, max, min).

Solution: Non-correlated scalar subquery in WHERE clause.

```
SELECT name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

Pattern 2 — Department-Level Benchmark (Correlated)

Problem: Compare each row against its own group's metric.

Solution: Correlated scalar subquery referencing the outer row's group key.

```
SELECT e.name, e.salary, e.dept_id
FROM employees e
WHERE e.salary = (
    SELECT MAX(salary) FROM employees
    WHERE dept_id = e.dept_id
);
-- Returns the top earner in each department
```

Pattern 3 — Existence Filter (Semi-Join)

Problem: Keep only rows that have at least one matching row elsewhere.

Solution: EXISTS with a correlated subquery — stops at first match.

```
SELECT c.*
FROM customers c
WHERE EXISTS (
    SELECT 1 FROM orders o
    WHERE o.customer_id = c.id AND o.status = 'completed'
);
```

Pattern 4 — Absence Filter (Anti-Join)

Problem: Find rows that have NO corresponding row in another table.

Solution: NOT EXISTS — NULL-safe, fastest anti-join method.

```
SELECT p.* FROM products p
WHERE NOT EXISTS (
    SELECT 1 FROM order_items oi WHERE oi.product_id = p.id
);
-- Products that have NEVER been ordered
```

Pattern 5 — Top-N Per Group

Problem: Get the N best rows for each group (e.g., top 3 orders per customer).

Solution: Correlated subquery counting how many rows are 'better' than current.

```
SELECT o.*
FROM orders o
WHERE (
    SELECT COUNT(*) FROM orders o2
    WHERE o2.customer_id = o.customer_id
    AND o2.total > o.total
) < 3
ORDER BY customer_id, total DESC;
-- Better in modern SQL: ROW_NUMBER() OVER (...) <= 3
```

Pattern 6 — Dynamic IN List

Problem: Filter using a set produced by a query rather than a hard-coded list.

Solution: Subquery in IN clause returning a dynamic list of valid IDs.

```
-- Orders from VIP customers (segment determined dynamically)
SELECT * FROM orders
WHERE customer_id IN (
    SELECT id FROM customers
    WHERE lifetime_value > 10000 AND active = 1
);
```

Pattern 7 — Aggregation Before Filtering (HAVING with subquery)

Problem: Filter aggregated groups against a value from another query.

Solution: Scalar subquery inside HAVING clause.

```
-- Departments with avg salary above the company average
SELECT dept_id, AVG(salary) AS dept_avg
FROM employees
GROUP BY dept_id
HAVING AVG(salary) > (SELECT AVG(salary) FROM employees);
```

Pattern 8 — Running Total with Correlated Subquery

Problem: Calculate a running sum ordered by a sequence column.

Solution: Correlated subquery summing all rows up to the current one.

```
SELECT s.sale_date, s.amount,
       (SELECT SUM(s2.amount) FROM sales s2
        WHERE s2.sale_date <= s.sale_date) AS running_total
FROM sales s
ORDER BY s.sale_date;

-- Modern approach: SUM(amount) OVER (ORDER BY sale_date)
```

Pattern 9 — Replace Missing Values from Another Table

Problem: If a value is missing in one table, fall back to a default from another.

Solution: COALESCE with a scalar subquery as the fallback expression.

```
SELECT o.id,
       COALESCE(
         cp.price,
         (SELECT price FROM default_prices WHERE product_id = o.product_id)
       ) AS effective_price
FROM orders o
LEFT JOIN customer_prices cp
  ON cp.customer_id = o.customer_id
 AND cp.product_id = o.product_id;
```

Pattern 10 — Compute Percentile / Rank via Subquery

Problem: Rank each row within its group without a window function.

Solution: Correlated COUNT of rows that rank higher than the current row.

```
SELECT  e.name, e.salary,
        1 + (SELECT COUNT(*) FROM employees e2
              WHERE  e2.dept_id = e.dept_id
                  AND e2.salary > e.salary) AS dept_rank
FROM    employees e
ORDER BY e.dept_id, dept_rank;
```

Pattern 11 — Insert Derived Data

Problem: Copy processed/transformed data into a new table in one statement.

Solution: INSERT INTO ... SELECT with a subquery or CTE as the source.

```
INSERT INTO customer_snapshots (customer_id, snapshot_date, ltv)
SELECT  customer_id,
        CURRENT_DATE,
        SUM(total)
FROM    orders
WHERE   status = 'completed'
GROUP BY customer_id;
```

Pattern 12 — Update from Another Table

Problem: Update column values in one table using computed values from another.

Solution: UPDATE ... SET with correlated scalar subquery.

```
UPDATE  products p
SET     p.units_sold = (
        SELECT COALESCE(SUM(quantity), 0)
        FROM   order_items oi
        WHERE  oi.product_id = p.id
      )
WHERE   p.active = 1;
```

Pattern 13 — Conditional Aggregation via Subquery

Problem: Produce multiple aggregates for different conditions without PIVOT.

Solution: Multiple scalar subqueries in SELECT, each with a different WHERE.

```
SELECT  dept_id,
        (SELECT COUNT(*) FROM employees e2
         WHERE e2.dept_id = e.dept_id AND e2.level='Senior') AS seniors,
        (SELECT COUNT(*) FROM employees e2
         WHERE e2.dept_id = e.dept_id AND e2.level='Junior') AS juniors
FROM    employees e
GROUP BY dept_id;
-- Modern: COUNT(CASE WHEN level='Senior' THEN 1 END) is faster
```

Pattern 14 — Deduplication via Subquery

Problem: Delete or identify duplicate rows, keeping only one per group.

Solution: NOT IN or NOT EXISTS subquery returning the IDs to keep.

```
-- Keep only the latest record per email
DELETE FROM users
WHERE id NOT IN (
    SELECT MAX(id)
    FROM   users
    GROUP BY email
);
```

Pattern 15 — Pagination with Offset Subquery

Problem: Efficiently paginate large result sets using a keyset approach.

Solution: Subquery to find the last-seen ID, then filter forward.

```
-- Keyset / cursor-based pagination (faster than OFFSET)
SELECT * FROM products
WHERE  id > (
    SELECT id FROM products
    ORDER BY id LIMIT 1 OFFSET 99
)
ORDER BY id LIMIT 20;
-- Or simply: WHERE id > :last_seen_id ORDER BY id LIMIT 20
```

Pattern 16 — Find Nth Highest Value

Problem: Retrieve the Nth highest (or lowest) value from a set.

Solution: Subquery counting rows with a higher value, then filtering to N-1.

```
-- 3rd highest salary
SELECT DISTINCT salary
FROM   employees e1
WHERE  2 = (
        SELECT COUNT(DISTINCT salary)
        FROM   employees e2
        WHERE  e2.salary > e1.salary
      );
-- N=3: change 2 to N-1
```

Pattern 17 — Flag Rows Matching a Subquery

Problem: Add a boolean column indicating whether a row meets a complex condition.

Solution: CASE + EXISTS scalar subquery to generate a flag column.

```
SELECT  p.name,
        p.price,
        CASE WHEN EXISTS (
            SELECT 1 FROM promotions pr
            WHERE  pr.product_id = p.id
            AND    pr.active = 1
        ) THEN 'ON SALE' ELSE 'Regular' END AS promo_status
FROM    products p;
```

Pattern 18 — Pivot Rows to Columns via CTE

Problem: Transform a vertical key-value table into a wide columnar format.

Solution: CTE to pre-filter, then conditional aggregation to pivot.

```
WITH attrs AS (
    SELECT product_id, attr_name, attr_value
    FROM   product_attributes
)
SELECT  product_id,
        MAX(CASE WHEN attr_name='color' THEN attr_value END) AS color,
        MAX(CASE WHEN attr_name='size' THEN attr_value END) AS size,
        MAX(CASE WHEN attr_name='weight' THEN attr_value END) AS weight
FROM    attrs
GROUP BY product_id;
```

Pattern 19 — Date Series Gap Detection

Problem: Find missing dates in a time series of events or records.

Solution: CTE-generated date series LEFT JOINed to actuals, filtering NULLs.

```
WITH RECURSIVE date_series AS (  
    SELECT '2024-01-01'::DATE AS d  
    UNION ALL  
    SELECT d + INTERVAL '1 day' FROM date_series WHERE d < '2024-01-31'  
)  
SELECT ds.d AS missing_date  
FROM   date_series ds  
WHERE  NOT EXISTS (  
    SELECT 1 FROM daily_sales s WHERE s.sale_date = ds.d  
)  
);
```

Pattern 20 — Multi-Step Business Rule Enforcement

Problem: Apply a complex business rule that requires multiple data lookups.

Solution: Chained CTEs, each encoding one rule step, composed in the final SELECT.

```
WITH  
eligible AS (  
    SELECT customer_id FROM customers  
    WHERE  account_age_days > 90 AND credit_score > 650  
)  
high_value AS (  
    SELECT customer_id FROM orders  
    WHERE  total > 500 AND created_at > CURRENT_DATE - 30  
)  
not_flagged AS (  
    SELECT customer_id FROM customers  
    WHERE  fraud_flag = 0  
)  
SELECT c.name, c.email  
FROM   customers c  
WHERE  c.id IN (SELECT customer_id FROM eligible)  
AND    c.id IN (SELECT customer_id FROM high_value)  
AND    c.id IN (SELECT customer_id FROM not_flagged);
```

*Classic Pitfalls & How to Avoid Them***! Pitfall 1 — Scalar Subquery Returning Multiple Rows**

Problem: A scalar subquery (in SELECT or WHERE =) that returns more than one row causes a runtime error: 'subquery returned more than one row'.

Fix: Add TOP 1 / LIMIT 1, or use MAX/MIN/ANY to guarantee a single value. If multiple values are valid, use IN instead of =.

! Pitfall 2 — NOT IN with NULL Values

Problem: If the subquery returns even one NULL, NOT IN evaluates every comparison to UNKNOWN — resulting in zero rows returned with no error or warning.

Fix: Always use NOT EXISTS (NULL-safe) or add WHERE col IS NOT NULL inside the NOT IN subquery. Make this a code review rule.

! Pitfall 3 — Correlated Subquery N+1 Performance

Problem: A correlated subquery in SELECT or WHERE executes once per outer row. On a table with 1M rows, this is 1M additional queries — catastrophic.

Fix: Replace with a window function (AVG OVER, SUM OVER) or a pre-aggregated JOIN. Always EXPLAIN correlated queries on large datasets.

! Pitfall 4 — Forgetting to Alias Derived Tables

Problem: In MySQL and some other engines, every derived table in the FROM clause MUST have an alias. Missing alias = syntax error.

Fix: Always alias every subquery: FROM (SELECT ...) AS my_alias. Make it a habit even in databases that don't strictly require it.

! Pitfall 5 — Reusing a CTE Multiple Times (Materialisation)

Problem: Some engines (e.g., PostgreSQL before v12) re-execute a CTE each time it is referenced. If referenced 3 times, it runs 3 times.

Fix: Use a temp table or materialised CTE (MATERIALIZED keyword in PG12+) for expensive CTEs referenced more than once.

! Pitfall 6 — Infinite Recursion in Recursive CTE

Problem: A recursive CTE without a proper termination condition loops forever or until the engine's recursion limit is reached.

Fix: Always include MAXRECURSION (SQL Server) or check your engine's limit. Add a depth counter and filter on depth < 100 as a safety guard.

! Pitfall 7 — Subquery in WHERE Instead of JOIN Causing Full Scan

Problem: WHERE id IN (SELECT ...) can force the engine to execute the inner query and materialise the full result before filtering.

Fix: Prefer EXISTS for existence checks. For large sets, rewrite as a JOIN with an index on the join key. Compare EXPLAIN plans.

! Pitfall 8 — Ambiguous Column References in Nested Subqueries

Problem: A column name in a deeply nested subquery might resolve to the outer query's table instead of the intended inner table — silently wrong results.

Fix: Always prefix every column with its table alias at every nesting level. Use distinct alias names across all levels (avoid reusing e, t, etc.).

! Pitfall 9 — CTE Not Visible Across Separate Statements

Problem: A CTE defined in one statement cannot be referenced in a subsequent statement. It exists only for that one query block.

Fix: If you need cross-statement reuse, use a VIEW or a temp table. CTEs are query-scoped, not session-scoped.

! Pitfall 10 — ORDER BY Inside a Subquery / CTE

Problem: ORDER BY inside a subquery (not the outermost query) has no guaranteed effect in SQL standard. The engine may ignore it.

Fix: Never rely on ORDER BY inside a derived table or CTE to order the final result. Apply ORDER BY only in the outermost SELECT.

Subquery Optimisation Rules

- **Push filters down:** filter inside the subquery, not just in the outer query — reduces the result set early.
- **Avoid SELECT *** in subqueries — select only the columns you need to minimise data transfer.
- **Replace correlated subqueries** with window functions whenever possible (AVG OVER, RANK OVER, SUM OVER).
- **Replace IN (large subquery)** with EXISTS — EXISTS short-circuits at first match.
- **Materialise expensive CTEs** referenced more than once into temp tables.
- **Index the columns** referenced in correlated subquery WHERE clauses.
- **Use EXPLAIN / EXPLAIN ANALYZE** on every query with a subquery before deploying.
- **Avoid functions on columns** inside subquery WHERE clauses — they prevent index use.

Subquery vs Window Function vs JOIN

```
-- PROBLEM: salary vs dept average
-- Method A: Correlated subquery (slow on large data)
SELECT name, salary,
       (SELECT AVG(salary) FROM employees e2 WHERE e2.dept_id = e.dept_id) AS dept_avg
FROM employees e;
-- Method B: Window function (BEST - single pass)
SELECT name, salary,
       AVG(salary) OVER (PARTITION BY dept_id) AS dept_avg
FROM employees;
-- Method C: JOIN to pre-aggregated subquery (good fallback)
SELECT e.name, e.salary, d.dept_avg
FROM employees e
JOIN (SELECT dept_id, AVG(salary) AS dept_avg
      FROM employees GROUP BY dept_id) d
ON e.dept_id = d.dept_id;
```

Quick Decision Guide

Situation	Best Approach
Filter by a single computed value	WHERE col = (scalar subquery)
Filter by a set of values	WHERE col IN (subquery) or EXISTS

Find rows with no match	NOT EXISTS or LEFT JOIN IS NULL
Add computed column from another table	SELECT (scalar subquery) or JOIN
Complex multi-step logic	Chained CTEs
Group-level metric vs row metric	Window function (OVER PARTITION BY)
Hierarchical / recursive data	Recursive CTE (WITH RECURSIVE)
Aggregate before joining	Derived table or CTE + JOIN
Reuse same result set multiple times	CTE (once) or Temp Table (large)
Existence check only (no columns needed)	EXISTS / NOT EXISTS
Top-N per group	LATERAL / ROW_NUMBER() OVER()

★ Senior Dev's Parting Words on Subqueries

1. Use CTEs — they are free in terms of performance and priceless in terms of readability. 2. Never use NOT IN without guarding against NULLs. 3. Replace correlated subqueries with window functions when possible. 4. Always run EXPLAIN before deploying any query with a subquery on a production-sized table. 5. The best subquery is sometimes no subquery — rethink your data model. 6. Compose queries like functions: one CTE = one clearly named step. 7. Test each CTE block independently before assembling the final query.